

# Unit 1

**1.Explain java constructor and constructor overloading, differentiate b/w constructor and method also write a java program to demonstrate constructor overloading.**

## 1) Java Constructor

According to your notes (page 11):

A **constructor** is:

- A block of code similar to a method.
- Called when an instance of a class is created.
- Used to initialize the object.
- Memory for the object is allocated when the constructor is called.
- Every time an object is created using the new keyword, at least one constructor is called.

### Rules for creating Java constructor:

- Constructor name must be same as class name.
- It must not have any explicit return type.
- It cannot be abstract, static, final, or synchronized.

### Types of Constructors (page 12):

1. **Default constructor (no-arg constructor)**
  2. **Parameterized constructor**
- 

## 2) Constructor Overloading

From page 15:

**Constructor overloading** is a technique of having more than one constructor in a class with different parameter lists (either different number of parameters or different type of parameters).

- Each constructor performs a different task.
- They are differentiated by the compiler based on number and type of parameters.

It improves readability and flexibility of object creation.

---

### 3) Difference Between Constructor and Method

(Based on page 20 of your PDF)

| Constructor                                              | Method                                           |
|----------------------------------------------------------|--------------------------------------------------|
| Used to initialize the state of an object                | Used to expose the behavior of an object         |
| Must not have a return type                              | Must have a return type                          |
| Invoked implicitly when object is created                | Invoked explicitly                               |
| Compiler provides default constructor if none is written | Method is not provided by compiler               |
| Constructor name must be same as class name              | Method name may or may not be same as class name |

### 4) Java Program to Demonstrate Constructor Overloading

(Based on example given in page 16)

```
class Student {  
    int id;  
    String name;  
    int age;  
  
    // Constructor with two parameters  
    Student(int i, String n) {  
        id = i;  
        name = n;  
    }  
  
    // Constructor with three parameters  
    Student(int i, String n, int a) {  
        id = i;  
        name = n;  
        age = a;  
    }  
}
```

```

    }

    void display() {
        System.out.println(id + " " + name + " " + age);
    }

    public static void main(String args[]) {
        Student s1 = new Student(111, "Karan");
        Student s2 = new Student(222, "Aryan", 25);

        s1.display();
        s2.display();
    }
}

```

### Output:

```

111 Karan 0
222 Aryan 25

```

(For first object, age is default value 0 because that constructor does not initialize age.)

## 2.explain the use of 'this' keyword in java and write a java program to demonstrate all possible uses of 'this' keyword

### Use of this Keyword in Java

this is a reference variable that refers to the **current object**.

It is used in the following ways:

#### 1) To refer current class instance variable

When local variables (parameters) and instance variables have the same name, this distinguishes them.

Example:

```

this.rollno = rollno;

```

## 2) To invoke current class method

You can invoke current class method using `this.methodName()`.

If not written explicitly, the compiler automatically adds `this`.

Example:

```
this.display();
```

## 3) To invoke current class constructor – `this()`

Used for constructor chaining.

Rule:

Call to `this()` must be the **first statement** in the constructor.

Example:

```
this(10);
```

## 4) To pass current object as argument in a method

You can pass `this` as an argument while calling a method.

Example:

```
m(this);
```

## 5) To pass current object as argument in constructor call

You can pass `this` to another constructor or class constructor.

Example:

```
B b = new B(this);
```

## 6) To return current class instance from method

A method can return `this`.

Example:

```
return this;
```

## Java Program Demonstrating All Uses of this

```
class Demo {

    int id;
    String name;

    // 1) this to refer instance variables
    Demo(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // 3) this() to invoke constructor (constructor chaining)
    Demo() {
        this(100, "Default");
        System.out.println("Default constructor called");
    }

    // 2) this to invoke method
    void show() {
        System.out.println("ID: " + id + " Name: " + name);
    }

    void display() {
        this.show(); // invoking current class method
    }

    // 4) this to pass as argument in method
    void method(Demo obj) {
        System.out.println("Object received with ID: " + obj.id);
    }

    void callMethod() {
        method(this);
    }

    // 6) this to return current class instance
    Demo getObject() {
        return this;
    }

    public static void main(String args[]) {
```

```

// Constructor chaining
Demo d1 = new Demo();

d1.display(); // invoke method
d1.callMethod(); // pass this as argument

// 5) this to pass as argument in constructor
Test t = new Test(d1);

// return current object
d1.getObject().show();
}
}

// 5) Passing this in constructor
class Test {
    Demo obj;

    Test(Demo obj) {
        this.obj = obj;
        System.out.println("Constructor received ID: " + obj.id);
    }
}

```

### Output (Conceptual)

```

Default constructor called
ID: 100 Name: Default
Object received with ID: 100
Constructor received ID: 100
ID: 100 Name: Default

```

## 3. Explain java IOStreams and differentiate b/w ByteStream and CharacterStream , write a java program to copy content from one file to another.

### Java I/O Streams

Java I/O (Input/Output) Streams are used to perform reading and writing operations on data.

A **stream** is a flow of data:

- **Input Stream** → reads data from source (file, keyboard, network, etc.)
- **Output Stream** → writes data to destination

Java provides the java.io package for handling I/O operations.

There are two main types of streams in Java:

1. **Byte Streams**
2. **Character Streams**

## 1. Byte Streams

Byte streams are used to handle **binary data** (images, audio, video, pdf, etc.).

They work with **8-bit bytes**.

Main classes:

- InputStream
- OutputStream
- FileInputStream
- FileOutputStream

### Small Example (Byte Stream)

```
import java.io.FileInputStream;

class ByteReadExample {
    public static void main(String args[]) throws Exception {
        FileInputStream fin = new FileInputStream("input.txt");

        int data;
        while ((data = fin.read()) != -1) {
            System.out.print((char)data);
        }

        fin.close();
    }
}
```

## 2. Character Streams

Character streams are used to handle **text data**.

They work with **16-bit Unicode characters**.

Main classes:

- Reader
- Writer
- FileReader
- FileWriter

### Small Example (Character Stream)

```
import java.io.FileReader;

class CharReadExample {
    public static void main(String args[]) throws Exception {
        FileReader fr = new FileReader("input.txt");

        int ch;
        while ((ch = fr.read()) != -1) {
            System.out.print((char)ch);
        }

        fr.close();
    }
}
```

### Difference Between Byte Stream and Character Stream

| Byte Stream                     | Character Stream                     |
|---------------------------------|--------------------------------------|
| Handles binary data             | Handles text data                    |
| Works with 8-bit bytes          | Works with 16-bit Unicode characters |
| Uses InputStream & OutputStream | Uses Reader & Writer                 |
| Suitable for images, audio, pdf | Suitable for text files              |
| Example: FileInputStream        | Example: FileReader                  |



## Java Program to Copy Content from One File to Another

### ✓ Using Byte Stream (Simple & Small)

```
import java.io.FileInputStream;
import java.io.FileOutputStream;

class FileCopyByte {
    public static void main(String args[]) throws Exception {

        FileInputStream fin = new FileInputStream("source.txt");
        FileOutputStream fout = new FileOutputStream("destination.txt");

        int data;

        while ((data = fin.read()) != -1) {
            fout.write(data);
        }

        fin.close();
        fout.close();

        System.out.println("File copied successfully using Byte Stream.");
    }
}
```

### ✓ Using Character Stream (For Text Files)

```
import java.io.FileReader;
import java.io.FileWriter;

class FileCopyChar {
    public static void main(String args[]) throws Exception {

        FileReader fr = new FileReader("source.txt");
        FileWriter fw = new FileWriter("destination.txt");

        int ch;

        while ((ch = fr.read()) != -1) {
            fw.write(ch);
        }

        fr.close();
    }
}
```

```
        fw.close();

        System.out.println("File copied successfully using Character Stream.");
    }
}
```

### Important Exam Points

- Byte streams → binary data
- Character streams → text data
- Always close streams after use
- read() returns -1 at end of file

## 4.Explain serialization and deserialization in java with an example program to serial and deserial a Student object.

### Serialization and Deserialization in Java

#### Serialization

**Serialization** is the process of converting an object into a byte stream so that it can be:

- Stored in a file
- Sent over a network
- Saved in memory

In Java, serialization is done using:

- Serializable interface (marker interface)
- ObjectOutputStream class

### Important Points:

- The class must implement Serializable.
- It is a marker interface (no methods).
- transient keyword is used to skip variables from serialization.

#### Deserialization

**Deserialization** is the reverse process of serialization.

It converts the byte stream back into an object.

In Java, deserialization is done using:

- `ObjectInputStream` class
- 

## Java Program: Serialize and Deserialize a Student Object

### Step 1: Create Student Class (Serializable)

```
import java.io.Serializable;

class Student implements Serializable {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    void display() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
    }
}
```

### Step 2: Serialize and Deserialize

```
import java.io.*;

class SerializeDemo {
    public static void main(String args[]) {

        try {

            // Serialization
            Student s1 = new Student(101, "Shashi");

            FileOutputStream fos = new FileOutputStream("student.txt");
            ObjectOutputStream oos = new ObjectOutputStream(fos);

            oos.writeObject(s1);
            oos.close();
        }
    }
}
```

```

        fos.close();

        System.out.println("Object Serialized Successfully");

        // Deserialization
        FileInputStream fis = new FileInputStream("student.txt");
        ObjectInputStream ois = new ObjectInputStream(fis);

        Student s2 = (Student) ois.readObject();
        ois.close();
        fis.close();

        System.out.println("Object Deserialized Successfully");
        s2.display();

    } catch (Exception e) {
        System.out.println(e);
    }
}
}

```

## Output

```

Object Serialized Successfully
Object Deserialized Successfully
ID: 101
Name: Shashi

```

## Flow Explanation

1. Student object is created.
2. writeObject() converts object into byte stream and stores in file.
3. readObject() reads byte stream from file.
4. Object is reconstructed in memory.

## Key Exam Points

- Serialization → Object → Byte Stream
- Deserialization → Byte Stream → Object
- Must implement Serializable
- Use ObjectOutputStream and ObjectInputStream
- Cast required during deserialization

## 5. Explain nested class in java , describe static nested class and inner class with suitable example.

### Nested Class in Java

A **nested class** is a class defined inside another class.

In Java, nested classes are used to:

- Group logically related classes
- Increase encapsulation
- Improve readability and maintainability

There are two main types of nested classes:

1. **Static Nested Class**
2. **Inner Class (Non-static Nested Class)**

### 1. Static Nested Class

A **static nested class** is declared with the static keyword.

#### Key Points:

- It cannot access non-static members of the outer class directly.
- It can access only static members of the outer class.
- It does not require an object of the outer class to be created.

#### Syntax:

```
class Outer {  
    static class Nested {  
        // code  
    }  
}
```

#### Example: Static Nested Class

```
class Outer {  
  
    static int data = 100;  
  
    static class Nested {  
        void display() {
```

```

        System.out.println("Static data: " + data);
    }
}

public static void main(String args[]) {
    Outer.Nested obj = new Outer.Nested();
    obj.display();
}
}

```

### Output:

Static data: 100

Here:

- Nested class accesses static variable data.
- No object of Outer class is created.

## 2. Inner Class (Non-static Nested Class)

An **inner class** is declared without the static keyword.

### Key Points:

- It can access both static and non-static members of outer class.
- Requires outer class object to create inner class object.

### Syntax:

```

class Outer {
    class Inner {
        // code
    }
}

```

### Example: Inner Class

```

class Outer {

    int data = 200;

    class Inner {

```

```

    void display() {
        System.out.println("Non-static data: " + data);
    }
}

public static void main(String args[]) {

    Outer outerObj = new Outer();    // outer object
    Outer.Inner innerObj = outerObj.new Inner(); // inner object

    innerObj.display();
}
}

```

### Output:

Non-static data: 200

Here:

- Inner class accesses non-static variable data.
- Outer object is required to create inner object.

### Difference Between Static Nested Class and Inner Class

| Static Nested Class                                            | Inner Class                                   |
|----------------------------------------------------------------|-----------------------------------------------|
| Declared with static keyword                                   | Declared without static                       |
| Cannot access non-static members directly                      | Can access all members                        |
| Does not need outer class object                               | Requires outer class object                   |
| Created as <code>Outer.Nested obj = new Outer.Nested();</code> | Created as <code>outerObj.new Inner();</code> |

### Important Exam Points

- Nested class = class inside another class
- Static nested class → behaves like static member
- Inner class → associated with outer class object
- Used for logical grouping and better encapsulation

## 6.Explain 'final' keyword in java with example

### final Keyword in Java

The final keyword in Java is used to **restrict** the user.

It can be applied to:

1. Variable
  2. Method
  3. Class
- 

### 1. Final Variable

When a variable is declared as final, its value **cannot be changed** once initialized.

#### Example: Final Variable

```
class FinalVariableDemo {  
    public static void main(String args[]) {  
  
        final int x = 10;  
        System.out.println("Value of x: " + x);  
  
        // x = 20; // Error: cannot change value of final variable  
    }  
}
```

#### Output:

Value of x: 10

- ✓ Used to create constants.
  - ✓ Once assigned, value cannot be modified.
- 

### 2. Final Method

When a method is declared as final, it **cannot be overridden** in a subclass.



## Example: Final Method

```
class Parent {  
  
    final void show() {  
        System.out.println("This is final method");  
    }  
}  
  
class Child extends Parent {  
  
    // void show() { } // Error: cannot override final method  
  
    void display() {  
        System.out.println("Child class method");  
    }  
}  
  
class Test {  
    public static void main(String args[]) {  
        Child obj = new Child();  
        obj.show();  
        obj.display();  
    }  
}
```

## Output:

This is final method

Child class method

✓ Used to prevent method overriding.

## 3. Final Class

When a class is declared as final, it **cannot be inherited**.

## Example: Final Class

```
final class Demo {  
    void show() {  
        System.out.println("Final class method");  
    }  
}
```

```
}  
}
```

```
// class Test extends Demo { } // Error: cannot inherit final class
```

```
class Main {  
    public static void main(String args[]) {  
        Demo obj = new Demo();  
        obj.show();  
    }  
}
```

## Output:

Final class method

✓ Used to prevent inheritance.

---

## Summary of final Keyword

| Applied To     | Effect                  |
|----------------|-------------------------|
| Final Variable | Value cannot be changed |
| Final Method   | Cannot be overridden    |
| Final Class    | Cannot be inherited     |

## Important Exam Points

- final = restriction keyword
- Final variable → constant
- Final method → prevents overriding
- Final class → prevents inheritance
- Frequently used in security and immutable classes

## 7. write a short note on access modifier in java with exaple

### Access Modifiers in Java

**Access modifiers** in Java are used to control the visibility (accessibility) of classes, variables, methods, and constructors.

They help in **data hiding and encapsulation**.

Java provides four types of access modifiers:

1. private
  2. default (no modifier)
  3. protected
  4. public
- 

## 1. Private

- Accessible **only within the same class**.
- Cannot be accessed outside the class.

### Example:

```
class Demo {  
  
    private int x = 10;  
  
    private void show() {  
        System.out.println("Private Method");  
    }  
  
    public static void main(String args[]) {  
        Demo obj = new Demo();  
        System.out.println(obj.x);  
        obj.show();  
    }  
}
```

✓ Used for data hiding.

---

## 2. Default (No Modifier)

- Accessible **within the same package only**.
- If no modifier is specified, it is default.

```
class Test {  
  
    int y = 20; // default access  
  
}
```

---

## 3. Protected

- Accessible within the same package.
- Accessible outside the package through inheritance.

```
class Parent {  
  
    protected void display() {  
  
        System.out.println("Protected Method");  
  
    }  
  
}
```

---

## 4. Public

- Accessible **everywhere**.
- No restriction.

```
public class Example {  
  
    public int z = 30;  
  
}
```

---

## Access Level Summary

| Modifier  | Same Class | Same Package | Subclass (Other Package) | Other Package |
|-----------|------------|--------------|--------------------------|---------------|
| private   | ✓          | ✗            | ✗                        | ✗             |
| default   | ✓          | ✓            | ✗                        | ✗             |
| protected | ✓          | ✓            | ✓                        | ✗             |
| public    | ✓          | ✓            | ✓                        | ✓             |

## Important Points for Exam

- Access modifiers provide security.
- private → highest restriction.
- public → least restriction.
- Default access is package-level access.
- Used to implement encapsulation.

## 8. Write a short note on DataInputStream and DataOutputStream .

### DataInputStream and DataOutputStream in Java

DataInputStream and DataOutputStream are byte stream classes available in the java.io package.

They are used to read and write **primitive data types** (int, float, double, boolean, etc.) in a machine-independent way.

They work together:

- DataOutputStream → writes primitive data to a file.
- DataInputStream → reads primitive data from a file.

They are usually used with:

- FileInputStream
- FileOutputStream

---

## 1. DataOutputStream

It is used to write primitive data types to an output stream.

### Important Methods:

- `writeInt(int)`
- `writeFloat(float)`
- `writeDouble(double)`
- `writeBoolean(boolean)`
- `writeUTF(String)`

### Example (Writing Data)

```
import java.io.*;

class DataOutputExample {
    public static void main(String args[]) throws Exception {

        FileOutputStream fos = new FileOutputStream("data.txt");
        DataOutputStream dos = new DataOutputStream(fos);

        dos.writeInt(100);
        dos.writeDouble(45.5);
        dos.writeUTF("Java");

        dos.close();
        fos.close();

        System.out.println("Data Written Successfully");
    }
}
```

## 2. DataInputStream

It is used to read primitive data types from an input stream.

### Important Methods:

- `readInt()`
- `readFloat()`
- `readDouble()`
- `readBoolean()`
- `readUTF()`

### Example (Reading Data)

```

import java.io.*;

class DataInputExample {
    public static void main(String args[]) throws Exception {

        FileInputStream fis = new FileInputStream("data.txt");
        DataInputStream dis = new DataInputStream(fis);

        int num = dis.readInt();
        double d = dis.readDouble();
        String str = dis.readUTF();

        dis.close();
        fis.close();

        System.out.println("Number: " + num);
        System.out.println("Double: " + d);
        System.out.println("String: " + str);
    }
}

```

## Important Points

- Used for reading and writing **primitive data types**.
  - Data must be read in the **same order** it was written.
  - Part of **byte stream hierarchy**.
  - Commonly used in file handling and network communication.
- 

## Summary

| DataOutputStream           | DataInputStream           |
|----------------------------|---------------------------|
| Writes primitive data      | Reads primitive data      |
| Uses write methods         | Uses read methods         |
| Works with<br>OutputStream | Works with<br>InputStream |

# Unit-2

## 1.Explain Inheritance in java , Describe difference types of Inheritance with suitable example.

### Inheritance in Java

**Inheritance** is an OOP concept in which one class acquires the properties (variables) and behaviors (methods) of another class.

It is achieved using the extends keyword.

- The class that inherits → **Subclass / Child class**
- The class being inherited → **Superclass / Parent class**

### Syntax:

```
class Parent {  
    // properties and methods  
}
```

```
class Child extends Parent {  
    // additional properties and methods  
}
```

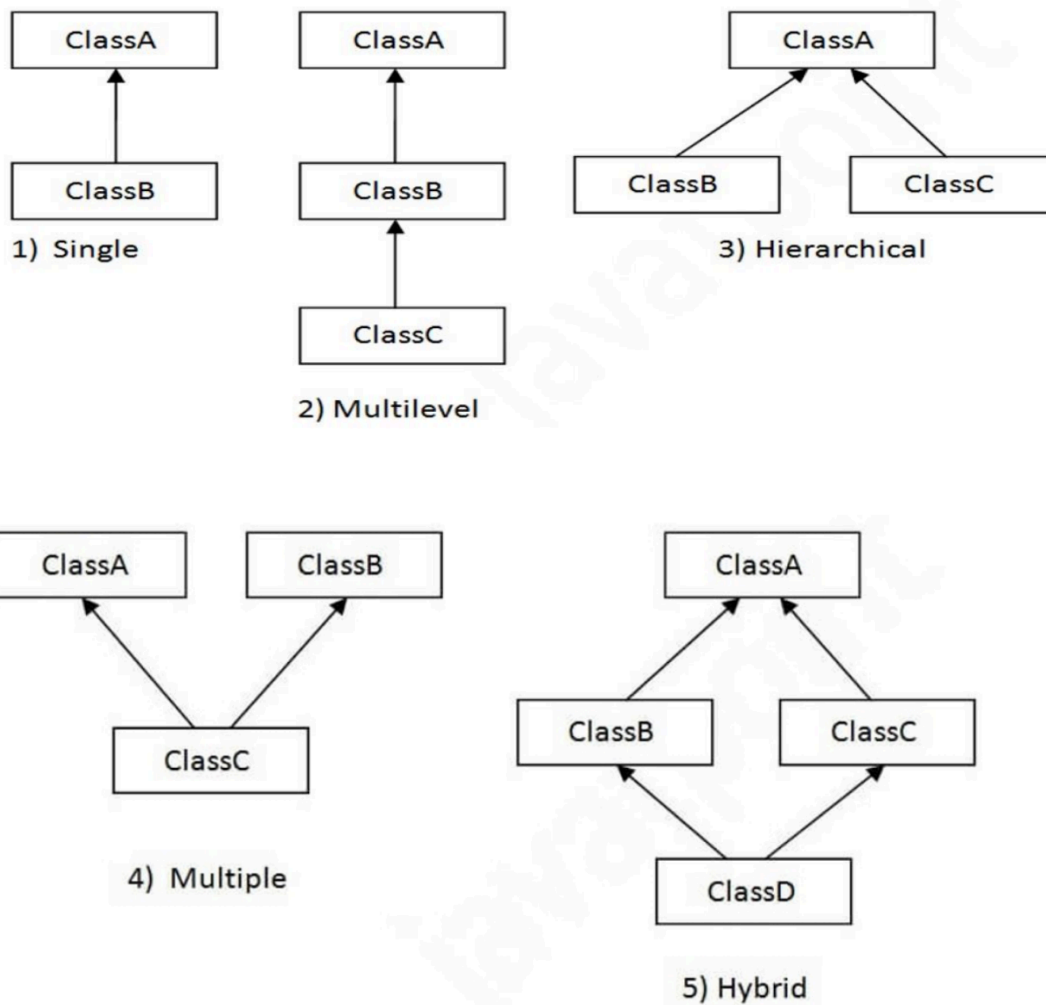
### Why Inheritance?

- Code reusability
- Method overriding
- Runtime polymorphism
- Logical hierarchy

### Types of Inheritance in Java

Java supports the following types:





1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance (through Interface only)
5. Hybrid Inheritance (through Interface only)

## 1. Single Inheritance

One child class inherits from one parent class.

### Example:

```
class Animal {  
    void eat() {  
        System.out.println("Eating...");  
    }  
}
```

```
class Dog extends Animal {
    void bark() {
        System.out.println("Barking...");
    }
}

class Test {
    public static void main(String args[]) {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

### Output:

Eating...  
Barking...

---

## 2. Multilevel Inheritance

A class inherits from a class which itself inherits another class.

### Example:

```
class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking...");
    }
}

class Puppy extends Dog {
    void weep() {
        System.out.println("Weeping...");
    }
}
```

```
class Test {  
    public static void main(String args[]) {  
        Puppy p = new Puppy();  
        p.eat();  
        p.bark();  
        p.weep();  
    }  
}
```

### Output:

Eating...  
Barking...  
Weeping...

---

## 3. Hierarchical Inheritance

Multiple classes inherit from a single parent class.

### Example:

```
class Animal {  
    void eat() {  
        System.out.println("Eating...");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barking...");  
    }  
}  
  
class Cat extends Animal {  
    void meow() {  
        System.out.println("Cat meowing...");  
    }  
}  
  
class Test {  
    public static void main(String args[]) {
```

```
Dog d = new Dog();
Cat c = new Cat();

d.eat();
c.eat();
}
}
```

---

#### 4. Multiple Inheritance (Through Interface Only)

Java does **not support multiple inheritance using classes** (to avoid ambiguity).

But it supports multiple inheritance using interfaces.

##### Example:

```
interface A {
    void show();
}
```

```
interface B {
    void display();
}
```

```
class C implements A, B {
    public void show() {
        System.out.println("Show method");
    }

    public void display() {
        System.out.println("Display method");
    }
}
```

```
class Test {
    public static void main(String args[]) {
        C obj = new C();
        obj.show();
        obj.display();
    }
}
```

---

## 5. Hybrid Inheritance

Combination of two or more types of inheritance.  
In Java, it is achieved only using interfaces.

---

### Summary Table

| Type         | Description                      |
|--------------|----------------------------------|
| Single       | One parent → One child           |
| Multilevel   | Parent → Child → Grandchild      |
| Hierarchical | One parent → Multiple children   |
| Multiple     | Multiple parents (via interface) |
| Hybrid       | Combination (via interface)      |

### Important Exam Points

- Inheritance is achieved using extends.
- Promotes code reusability.
- Java does not support multiple inheritance using classes.
- Interfaces allow multiple inheritance.

**2.Explain method overriding , Runtime polymorphism , Dynamic method dispatch , Differentiate between Method overloading and Method overriding and write a java program to demonstrate.**

### 1. Method Overriding

**Method Overriding** occurs when a subclass provides a specific implementation of a method that is already defined in its parent class.

## Conditions:

- Method name must be same.
- Parameters must be same.
- Inheritance must exist.
- Access modifier cannot be more restrictive.
- Overridden method cannot be final, static, or private.

## Example:

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

---

## 2. Runtime Polymorphism

**Runtime Polymorphism** is achieved through **method overriding**.

- Method call is resolved at runtime.
  - It depends on the object being referred to.
  - Achieved using inheritance and method overriding.
- 

## 3. Dynamic Method Dispatch

**Dynamic Method Dispatch** is the mechanism by which a call to an overridden method is resolved at runtime.

It works using:

- Upcasting (Parent reference → Child object)

Example:

```
Animal a = new Dog(); // upcasting
a.sound();           // decided at runtime
```

Here, even though reference is Animal, method of Dog is executed.

#### 4. Difference Between Method Overloading and Method Overriding

| Method Overloading                     | Method Overriding                       |
|----------------------------------------|-----------------------------------------|
| Same method name, different parameters | Same method name, same parameters       |
| Occurs within same class               | Occurs between parent and child         |
| Compile-time polymorphism              | Runtime polymorphism                    |
| No inheritance required                | Inheritance required                    |
| Return type may differ                 | Return type must be same (or covariant) |

#### 5. Java Program to Demonstrate Overriding, Runtime Polymorphism & Dynamic Method Dispatch

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
```

```
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
```

```
class Cat extends Animal {
    void sound() {
        System.out.println("Cat meows");
    }
}
```

```
class Test {  
    public static void main(String args[]) {  
  
        Animal a; // Parent reference  
  
        a = new Dog(); // Upcasting  
        a.sound();    // Calls Dog's method  
  
        a = new Cat(); // Upcasting  
        a.sound();    // Calls Cat's method  
    }  
}
```

## Output

Dog barks

Cat meows

## Explanation of Program

- Animal class has method sound().
  - Dog and Cat override sound().
  - Parent reference a refers to different child objects.
  - JVM decides at runtime which method to call.
  - This is **Runtime Polymorphism**.
  - The mechanism is called **Dynamic Method Dispatch**.
- 

## Important Exam Points

- Overriding = same method, same parameters.
- Overloading = same method name, different parameters.
- Runtime polymorphism achieved using overriding.
- Dynamic method dispatch = method call resolved at runtime.
- Upcasting is required for runtime polymorphism.

**3.Explain the concept of packages and sub packages in java ,  
Illustrate with suitable example.**

## Packages in Java



A **package** in Java is a namespace that groups related classes and interfaces together.

It is used to:

- Organize classes logically
- Avoid name conflicts
- Provide access protection
- Improve code maintainability

## Creating a Package

The package keyword is used at the top of a Java file.

### Syntax:

```
package packagename;
```

### Example:

```
package mypack;

public class Demo {
    public void display() {
        System.out.println("Inside mypack package");
    }
}
```

To use this class in another file:

```
import mypack.Demo;

class Test {
    public static void main(String args[]) {
        Demo obj = new Demo();
        obj.display();
    }
}
```

## Types of Packages

1. Built-in Packages (like java.util, java.io)
2. User-defined Packages

## Subpackage in Java

A **subpackage** is a package inside another package.

It helps in further categorizing classes.

### **Syntax:**

```
package mainpackage.subpackage;
```

## **Example of Subpackage**

### **Step 1: Create Subpackage Class**

```
package mypack.subpack;  
  
public class Message {  
    public void show() {  
        System.out.println("Inside subpackage");  
    }  
}
```

### **Step 2: Use Subpackage Class**

```
import mypack.subpack.Message;  
  
class Test {  
    public static void main(String args[]) {  
        Message obj = new Message();  
        obj.show();  
    }  
}
```

### **Directory Structure**

```
mypack/  
    Demo.java  
    subpack/  
        Message.java
```

### **Important Points**

- package statement must be first line in file.
- Folder structure must match package name.

- Use import keyword to access classes from other packages.
- Subpackage does NOT automatically inherit access from parent package.

### Difference Between Package and Subpackage

| Package               | Subpackage                     |
|-----------------------|--------------------------------|
| Top-level grouping    | Package inside another package |
| Example: mypack       | Example: mypack.subpack        |
| Independent namespace | Also independent namespace     |

### Exam Key Points

- Used for organizing classes.
- Avoids naming conflicts.
- Provides access control.
- Subpackage is treated as separate package.

### 4.Explain Exception Handling in java , describe try, catch, finally, throw and throws keyword.

### Exception Handling in Java

**Exception Handling** is a mechanism in Java to handle runtime errors so that the normal flow of the program is not terminated abruptly.

An **exception** is an unwanted or unexpected event that occurs during program execution.

Java handles exceptions using five keywords:

- try
  - catch
  - finally
  - throw
  - throws
-

## 1. try Block

The try block contains the code that may cause an exception.

### Syntax:

```
try {  
    // risky code  
}
```

### Example:

```
try {  
    int a = 10 / 0; // causes ArithmeticException  
}
```

## 2. catch Block

The catch block handles the exception thrown in the try block.

- It must follow the try block.
- Multiple catch blocks are allowed.

### Syntax:

```
catch(ExceptionType e) {  
    // handling code  
}
```

### Example:

```
try {  
    int a = 10 / 0;  
}  
catch(ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
}
```

## 3. finally Block

The finally block always executes whether an exception occurs or not.

Used for:

- Closing files
- Releasing resources

- Database connections

**Syntax:**

```
finally {  
    // cleanup code  
}
```

**Example:**

```
try {  
    int a = 10 / 2;  
}  
catch(Exception e) {  
    System.out.println(e);  
}  
finally {  
    System.out.println("This always executes");  
}
```

#### 4. throw Keyword

The throw keyword is used to explicitly throw an exception.

**Syntax:**

```
throw new ExceptionType("message");
```

**Example:**

```
class Test {  
    public static void main(String args[]) {  
  
        int age = 15;  
  
        if(age < 18) {  
            throw new ArithmeticException("Not eligible to vote");  
        }  
  
        System.out.println("Eligible");  
    }  
}
```

## 5. throws Keyword

The throws keyword is used to declare exceptions.

- It is used in method signature.
- It indicates that the method may throw an exception.

### Syntax:

```
returnType methodName() throws ExceptionType {  
}
```

### Example:

```
import java.io.*;  
  
class Test {  
    static void readFile() throws IOException {  
        FileReader fr = new FileReader("file.txt");  
    }  
  
    public static void main(String args[]) {  
        try {  
            readFile();  
        }  
        catch(IOException e) {  
            System.out.println("File not found");  
        }  
    }  
}
```

### Complete Example Using All Keywords

```
class ExceptionDemo {  
  
    static void check(int num) throws Exception {  
  
        if(num < 0) {  
            throw new Exception("Number is negative");  
        }  
        else {  
            System.out.println("Number is positive");  
        }  
    }  
}
```

```

public static void main(String args[]) {

    try {
        check(-5);
    }
    catch(Exception e) {
        System.out.println("Exception caught: " + e.getMessage());
    }
    finally {
        System.out.println("Program finished");
    }
}
}

```

### Difference Between throw and throws

| throw                                 | throws                      |
|---------------------------------------|-----------------------------|
| Used to explicitly throw an exception | Used to declare exception   |
| Used inside method                    | Used in method signature    |
| Followed by object                    | Followed by exception class |

### Important Exam Points

- try → risky code
- catch → handles exception
- finally → always executes
- throw → explicitly throw exception
- throws → declare exception

### 5. Write a java program to illustrate multiple catch blocks.

### Java Program to Illustrate Multiple Catch Blocks

**Multiple catch blocks** are used when a try block may generate different types of exceptions.

Each catch block handles a specific exception type.

### Important Points:

- A single try block can have multiple catch blocks.

- Catch blocks must be arranged from **most specific to most general**.
- Only one catch block executes for a thrown exception.

## Example Program

```
class MultipleCatchDemo {  
  
    public static void main(String args[]) {  
  
        try {  
            int a = 10;  
            int b = 0;  
  
            int result = a / b; // ArithmeticException  
  
            int arr[] = new int[5];  
            arr[10] = 50;      // ArrayIndexOutOfBoundsException  
  
            String str = null;  
            System.out.println(str.length()); // NullPointerException  
        }  
  
        catch (ArithmeticException e) {  
            System.out.println("Arithmetic Exception: Cannot divide by zero");  
        }  
  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array Index Out Of Bounds Exception");  
        }  
  
        catch (NullPointerException e) {  
            System.out.println("Null Pointer Exception");  
        }  
  
        catch (Exception e) {  
            System.out.println("General Exception");  
        }  
  
        finally {  
            System.out.println("Program execution completed");  
        }  
    }  
}
```



## Output

Arithmetic Exception: Cannot divide by zero

Program execution completed

## Explanation

- The first exception (10 / 0) causes an ArithmeticException.
- JVM immediately transfers control to the matching catch block.
- Remaining statements inside try are not executed.
- finally block executes in all cases.

## Key Exam Points

- Multiple catch blocks handle different exceptions.
- Order: Specific → General (Exception must be last).
- Only one catch block executes.
- finally always executes.